# TOURSAFE

## AI-Driven Emergency Response & Blockchain Identity Infrastructure

*Full Project Document — Design, Architecture & Execution Blueprint*


| | |
|---|---|
| **Project Team**<br>TriArch | **Team Members**<br>Vishal Lakshmikanthan, Sneha C & Madhu |
| **Institution**<br>Sri Sairam Engineering College, Chennai | **Department**<br>Computer Science & Engineering |
| **Document Type**<br>Final Year Project — Complete Blueprint | **Version**<br>v1.0 — June 2026 |


*"No traveler should have to choose between exploring the world and staying safe."*

# 0. Executive Summary

TourSafe is a comprehensive, proactive travel safety ecosystem engineered to fundamentally solve the "Golden Hour" emergency response crisis faced by international and domestic tourists worldwide. The system transitions travel safety from a purely reactive paradigm — one that acts only after a tragedy has escalated — into an AI-powered, silent guardian infrastructure that autonomously detects, verifies, and dispatches emergency resources before a victim can even reach for their phone.

The platform is architected around three integrated technological pillars: an AI anomaly detection engine built on a Sequence-to-Sequence LSTM Autoencoder for real-time crash and immobility detection; a Blockchain-Anchored Decentralized Identity (DID) layer deployed on the Polygon PoS network for instant, tamper-proof identity verification in the field; and a MERN-stack Business-to-Government (B2G) command dashboard with live incident heatmaps and automated e-FIR (electronic First Information Report) generation.

### Core Mission & Target Impact

TourSafe targets a 70% reduction in emergency response latency — compressing the identification-to-dispatch window from a fatal 20-30 minute lag to under 5 minutes. This preserves the bulk of the critical 'Golden Hour' for life-saving medical intervention. The initial rollout targets 5,000+ travelers across designated Safe Zones, directly aligning with UN Sustainable Development Goals SDG 8, SDG 11, and SDG 16.

This document serves as the definitive, AI-agent-ready architectural and execution blueprint for the TourSafe project. It is structured so that any development team, AI coding agent, or stakeholder can comprehend the full scope, then independently implement each module without ambiguity. No source code is included; instead, every module is described with sufficient technical precision that the implementation pathway is unambiguous.

# 1. Problem Statement & Vision

## 1.1 The Trust Gap in Global Travel Safety

Every year, millions of travelers — solo hikers, adventure tourists, international visitors — venture into unfamiliar territories carrying only the promise of wonder and the fragile assumption that someone will help if something goes wrong. This assumption is systematically and fatally broken. The global travel safety ecosystem operates on a deeply flawed, reactive logic: it provides insurance payouts after a tragedy, or relies on SOS buttons that are entirely useless if the victim is unconscious, panicking, or separated from their device.

The statistical reality is stark. Injuries are the leading cause of non-natural death for international travelers, accounting for up to 25% of all traveler fatalities and exceeding infectious diseases as a cause of death by a factor of up to 25 times. Yet the infrastructure designed to protect these individuals is critically fragmented, technologically stagnant, and geographically blind to the very remote zones where the highest-risk tourism activity occurs.

## 1.2 The Fatal Golden Hour Delay

In emergency medicine, the 'Golden Hour' is the window immediately following a traumatic injury during which prompt medical treatment is most likely to prevent death. Every minute of delayed response reduces a cardiac arrest or severe trauma victim's odds of survival by 7 to 10 percent. The current state of emergency response in tourism contexts destroys this window:

- Urban Emergency Medical Services (EMS) achieve a response coverage of 90% of calls in under 9 minutes — a benchmark that is already marginal for acute trauma.
- In rural and remote areas where adventure and nature tourism is concentrated, EMS response times are up to 3 times slower, with 1-in-10 encounters resulting in wait times of nearly 30 minutes just for help to arrive on scene.
- Beyond physical response delays, the identification and verification of an incapacitated tourist — who may be unconscious, speaking a foreign language, or carrying unfamiliar identity documents — introduces an additional 20-30 minute administrative lag before any medical protocol can even begin.
- The cumulative effect: a tourist in a remote area experiencing cardiac arrest or severe trauma may wait 45-60 minutes before receiving any medical care, reducing survival probability to near zero.

## 1.3 Root Causes of the Crisis

Three fundamental system failures perpetuate this crisis and define the design targets for TourSafe:

### 1.3.1 Predictive Blindness and Connectivity Gaps

Current safety systems lack any capacity for proactive, predictive monitoring. There is no AI-based anomaly detection infrastructure and no geo-fencing for high-risk zones. When tourists enter remote areas with poor or absent network coverage, standard GPS tracking drops entirely, leaving the traveler invisible to any monitoring system until a crisis is manually reported by a bystander or until the victim regains the capacity to initiate a call. This reactive design means the system is structurally incapable of protecting the most vulnerable travelers in the most dangerous locations.

### 1.3.2 The Global Identity Crisis

When an unconscious or incapacitated tourist is found by first responders, authorities face a bureaucratic and logistical nightmare. Globally, over 850 million people lack official, legal identification documents. Even among those who carry identification, there are over 6,000 unique, valid ID document types in global circulation — each with different formats, languages, and verification protocols. Cross-border data localization restrictions legally prevent local police and hospitals from accessing a foreign tourist's critical medical history, allergy records, blood type, or emergency contact information. The result is that life-saving treatment is delayed or compromised by administrative uncertainty.

### 1.3.3 Reliance on Legacy and Fragmented Systems

The existing emergency response infrastructure relies on paper-based reporting processes, manual data entry, phone-based dispatch coordination, and siloed, incompatible databases across police, EMS, and hospital systems. There is no automated pipeline that can simultaneously identify a victim, locate the nearest emergency resources, generate a standardized incident report, and dispatch multiple agencies within seconds. Every step in this chain introduces delay, error, and friction that directly costs lives.

## 1.4 The Opportunity: A ₹15.73 Lakh Crore Ecosystem Unprotected

India's tourism economy alone represents a market value of approximately ₹15.73 Lakh Crore, operating with tourists who are, in practice, protected by almost nothing. This is not merely a humanitarian crisis — it is a structural economic vulnerability. Traveler fear of safety incidents restricts tourism growth, inflates insurance premiums across the industry, damages destination reputations, and creates long-term economic stagnation for communities whose livelihoods depend on tourism. TourSafe's implementation directly addresses this by providing 'Safe Zone' certification for destinations, restoring traveler confidence and driving measurable economic growth.

# 2. Solution Architecture Overview

## 2.1 The TourSafe Paradigm: From Reactive to Proactive

TourSafe is not an emergency button. It is a continuous, autonomous, AI-powered safety layer that operates silently in the background of a traveler's journey, requiring zero active engagement from the user during a crisis. The system is designed to be most effective precisely when the user is least capable of acting — when they are unconscious, incapacitated, or in shock.

The architectural philosophy is built on three interlocking principles: First, the system must be proactive, detecting emergencies autonomously through sensor data rather than waiting for user input. Second, the system must be identity-sovereign, allowing any first responder anywhere in the world to instantly access critical traveler information without bureaucratic delay. Third, the system must be resilient, functioning effectively even in zero-connectivity remote environments by caching and queuing data locally until a signal is restored.

## 2.2 The Three-Layer Shield

| Layer | Description & Role |
|---|---|
| **Layer 1: The Watchful Guardian** | The AI Anomaly Engine. A Sequence-to-Sequence LSTM Autoencoder continuously processes 50Hz IMU sensor data from the traveler's mobile device. It autonomously detects crash signatures (massive G-force spikes) and immobility signatures (prolonged flatline matching Earth's static gravity in an isolated zone), triggering emergency protocols without any user action. |
| **Layer 2: The Bridge of Trust** | The Blockchain DID Identity Layer. Each traveler is issued a Self-Sovereign, tamper-proof Decentralized Identity (DID) anchored on the Polygon PoS blockchain. A secure QR code on the traveler's device allows any authorized first responder to instantly decrypt critical medical data and emergency contacts using Emergency Cryptographic Access Keys — no paperwork, no bureaucracy, no delay. |
| **Layer 3: The Command for Safety** | The B2G Dashboard & Geo-fencing Command Center. A MERN-stack dashboard with Mapbox GL JS visualization provides authorities with live telemetry, real-time incident heatmaps, and geo-fence breach alerts. Upon AI anomaly confirmation, the system automatically compiles and dispatches a standardized e-FIR containing GPS coordinates, medical profiles, and LSTM logs to the nearest police station and hospital nodes. |

## 2.3 System-Level Data Flow

At the highest level, data flows through TourSafe in a continuous, bidirectional loop. The traveler's mobile application is the primary sensor platform, collecting GPS coordinates at 1Hz and IMU

accelerometer data at 50Hz. This telemetry stream is transmitted in near-real-time to the FastAPI backend via WebSocket channels. The backend's ML inference engine evaluates each incoming window of sensor data. Under normal conditions, data is logged to MongoDB and GPS coordinates are cached in Redis. When the anomaly engine fires, the entire emergency response chain is triggered simultaneously: the authority dashboard receives a real-time WebSocket push alert, the Blockchain DID contract is queried to decrypt the victim's identity credentials, the e-FIR microservice compiles and dispatches the incident report, and the Haversine routing algorithm identifies and notifies the nearest emergency resources.

### Offline-First Architecture

A critical design requirement is that the system must not fail when network connectivity is lost — the exact scenario where the most dangerous tourism accidents occur. The mobile application implements a data-interceptor pattern: if a network ping times out, the telemetry payload is immediately encrypted using AES-256 and queued in a local SQLite database buffer. The moment connectivity is restored, an autocommit mechanism flushes the entire offline queue to the cloud server, ensuring zero loss of tracking data even after prolonged offline periods.

# 3. Technical Architecture — Component-by-Component

## 3.1 Client-Side Mobile Application

### 3.1.1 Platform & Framework

The TourSafe mobile application is built from scratch on React Native with TypeScript, using the bare React Native CLI workflow (not Expo managed). This deliberate choice of the bare workflow ensures full, unrestricted access to native device hardware — specifically the IMU accelerometer at 50Hz polling rates, background location services, and the device's secure hardware enclave for cryptographic key storage — without the abstraction constraints imposed by a managed environment. The choice of React Native enables a single TypeScript codebase to produce fully native builds for both iOS and Android, which is essential for maximizing traveler adoption across different device ecosystems. TypeScript is non-negotiable for this application because the core data structures — GPS coordinate arrays, timestamped IMU telemetry packets, and cryptographic key objects — require compile-time type safety to prevent data corruption in life-critical operations. All development is performed in Visual Studio Code with the AnthropicAI extension and GitHub Copilot for AI-assisted development, and Android Studio is used for Android emulator configuration, device debugging, and APK build validation during the Android-specific testing phases.

### 3.1.2 Core Mobile Libraries & Rationale

| Library / Module | Purpose & Technical Rationale |
|---|---|
| **React Native (TypeScript) + Expo** | Cross-platform native iOS/Android build from a single codebase using the bare React Native CLI. Direct native module access for hardware peripherals (50Hz IMU, background GPS, secure enclave). Developed in VS Code with AnthropicAI and GitHub Copilot; Android emulator phases run in Android Studio. |
| **TanStack Query** | Handles all server-state synchronization, including automatic background refetching, cache invalidation, and optimistic updates. Prevents stale data from corrupting the live telemetry view. |
| **Google Maps API + Turf.js** | Google Maps provides the base map rendering for the traveler's location awareness UI. Turf.js is a geospatial analysis library used exclusively for client-side computation of geo-fencing boundary checks against cached GeoJSON hazard zone data. This offline geo-fence checking means boundary violations can be detected even without a network signal. |
| **SQLite + AsyncStorage** | Dual-layer offline buffer. SQLite stores encrypted telemetry queue payloads for offline-first resilience. AsyncStorage stores lightweight session state and user preferences. Together, these ensure zero data loss in zero-connectivity scenarios. |
| **Redux Toolkit** | Manages the global application state, including the traveler's current safety status, active geo-fence alerts, connection state, and |

| | |
|---|---|
| | offline queue metrics. Provides a single source of truth for all UI components. |
| **Expo Sensors API** | Provides programmatic access to the device's Inertial Measurement Unit (IMU), specifically the accelerometer. Configured for 50Hz polling to capture the high-frequency sensor data required by the LSTM Autoencoder for accurate anomaly detection. |
| **Expo Location API** | Provides background GPS location tracking at 1Hz. Configured with foreground and background location permissions to ensure continuous tracking even when the app is not in the foreground — critical for protecting travelers who are actively engaged in outdoor activities. |
| **React Native Keychain / Expo SecureStore** | Handles secure local storage of the traveler's private cryptographic key for the DID wallet. Keys are stored in the device's secure enclave or hardware-backed keystore, never in plain memory. |
| **ethers.js** | The Ethereum/Polygon JavaScript library used for wallet generation, smart contract interaction, and transaction signing on the client side. Handles the asymmetric key generation for the DID vault and the QR code payload signing. |

### 3.1.3 Sensor Data Collection Pipeline

The mobile application's sensor collection pipeline operates as a background service. The accelerometer listener is initialized at application startup and configured to poll at a target rate of 50 samples per second (50Hz). Each raw sample is a three-axis vector (Ax, Ay, Az) representing acceleration forces along the device's local X, Y, and Z axes. These raw samples are accumulated in an in-memory ring buffer.

Every 3 seconds, the ring buffer is read to produce a sliding window of 150 data points (50Hz x 3 seconds). The window slides with a 50% overlap (1.5 seconds), meaning a new window is generated every 1.5 seconds. This overlap is critical because it ensures that sudden impact events cannot be missed by falling entirely between two non-overlapping windows. Each window is vectorized using the total acceleration magnitude formula and packaged as a telemetry payload alongside the current GPS coordinate and timestamp.

### 3.1.4 Offline Buffer & AES-256 Encryption Flow

The network interceptor is implemented as a middleware layer sitting between the telemetry producer and the WebSocket transmitter. Before every transmission attempt, the interceptor performs a lightweight ping to the FastAPI backend. If the ping returns a timeout error (indicating no network coverage), the interceptor immediately serializes the telemetry payload to a JSON string and encrypts it using AES-256-CBC with a session-derived symmetric key. The encrypted blob is then inserted as a row into the local SQLite queue table with a timestamp and a 'pending' status flag.

A background job polls the network status every 30 seconds. When connectivity is restored — indicated by a successful ping — the autocommit mechanism selects all rows with 'pending' status from the SQLite queue in chronological order and transmits them sequentially to the FastAPI backend via the WebSocket channel. Each successfully acknowledged row is updated to 'committed' status. This ensures that even if a traveler spends hours in a no-signal zone, the complete historical telemetry record is delivered intact to the server upon reconnection.

## 3.2 Backend & Real-Time Telemetry Engine

### 3.2.1 FastAPI (Python) — The ASGI Core

The backend is built exclusively on FastAPI, a modern Python web framework built on the ASGI (Asynchronous Server Gateway Interface) standard. The deliberate choice of FastAPI over alternatives like Django, Flask, or Java Spring Boot is driven by two converging technical requirements: the proximity to Python's machine learning ecosystem (TensorFlow, ONNX Runtime), and the framework's native support for high-concurrency asynchronous operations via the asyncio event loop.

Processing 50Hz sensor data from potentially thousands of concurrent travelers is an I/O-bound workload that would cause catastrophic thread-blocking in a synchronous framework. FastAPI's async architecture allows it to handle thousands of concurrent WebSocket connections and telemetry ingestion streams without spawning a new thread per connection, making it inherently scalable for this use case. All Pydantic data models for incoming telemetry packets, anomaly thresholds, and incident payloads are strictly typed to enforce data integrity at the API boundary.

### 3.2.2 WebSocket Telemetry Handler

The primary data ingestion endpoint is an asynchronous WebSocket handler mounted at the path /ws/telemetry/{traveler_id}. Unlike HTTP endpoints that follow a request-response cycle, this persistent WebSocket connection remains open for the duration of the traveler's session. The server receives incoming telemetry windows continuously, passes each window to the ML inference worker pool, and evaluates the reconstruction error. This bidirectional channel also allows the server to push configuration updates, geo-fence boundary data, and emergency notifications back to the client without the client needing to poll.

### 3.2.3 Redis — In-Memory GPS Cache

Redis serves a highly specific and narrowly scoped function in the TourSafe stack: it is the in-memory cache exclusively for the live GPS coordinates of all currently tracked travelers. The rationale is performance. The B2G authority dashboard needs to render the real-time positions of potentially thousands of travelers on a live map with sub-second update latency. Querying MongoDB for live positions at this frequency would be prohibitively slow and expensive. Redis stores each traveler's latest GPS coordinate as a key-value pair (traveler_id -> {lat, lng, timestamp}), providing sub-millisecond read latency for the dashboard's real-time map rendering queries. GPS data in Redis is set with a 5-minute TTL, ensuring the cache automatically evicts stale data for travelers who have gone offline.

### 3.2.4 MongoDB — Persistent Document Storage

MongoDB provides the persistent storage layer for all complex, semi-structured data that does not fit neatly into a relational schema. The primary collections include: the Traveler Profile collection (storing encrypted personal metadata, DID identifiers, and subscription records); the Telemetry Archive collection (storing processed, anonymized telemetry logs for heatmap generation after trip completion); the Incident Log collection (storing full, detailed records of every triggered anomaly event, including LSTM reconstruction error traces, GPS coordinates, and response timestamps); and the e-FIR Archive (storing copies of all generated electronic First Information Reports with their dispatch status). The schema-flexible nature of MongoDB is particularly well-suited for incident records, which vary in structure depending on the type of anomaly detected and the jurisdiction of the dispatch recipient.

### 3.2.5 Socket.io — Real-Time Authority Notification

Socket.io implements the persistent, bidirectional WebSocket event channels between the FastAPI backend and the MERN dashboard. The critical architectural decision to use Socket.io rather than HTTP polling for authority notifications is a direct consequence of the response latency target. If the dashboard polled the backend every 5 seconds, the maximum notification delay for an incident that occurred immediately after a poll would be 5 seconds — already approaching the limits acceptable for a life-critical system. Socket.io's event-driven push model means the millisecond the LSTM anomaly threshold is crossed, a structured incident packet is emitted to all connected authority dashboard clients subscribed to the relevant geographic zone, with propagation latency measured in tens of milliseconds rather than seconds.

## 3.3 Machine Learning Engine

### 3.3.1 The Sequence-to-Sequence LSTM Autoencoder

The intelligence core of TourSafe is a Sequence-to-Sequence Long Short-Term Memory (LSTM) Autoencoder, implemented using TensorFlow and Keras. This model architecture is chosen because human movement is fundamentally a sequential temporal phenomenon — the patterns of a person walking, sitting in a vehicle, or hiking a trail are not collections of independent data points but structured time-series with meaningful temporal dependencies that only LSTMs can capture.

The Autoencoder is trained exclusively on labeled normal activity data: walking, driving, sitting, and light hiking. The training objective is reconstruction: the encoder compresses the input telemetry sequence into a compact latent representation, and the decoder attempts to reconstruct the original sequence from this compressed form. After training on thousands of hours of normal activity data, the model becomes highly efficient at reconstructing normal movement patterns and correspondingly poor at reconstructing abnormal ones. This difference in reconstruction quality — quantified as the Mean Squared Error between the input sequence and its reconstruction — is the Reconstruction Error, and it becomes the primary anomaly signal.

**Why an Autoencoder Instead of a Supervised Classifier?**

A supervised classifier would require extensive labeled datasets of actual crash events and unconsciousness episodes from real travelers — data that is both ethically impossible to collect and statistically rare. The Autoencoder approach requires only normal activity data, which is abundant. Any event that deviates from the learned normal patterns — regardless of whether that specific deviation was ever seen in training — will produce a high reconstruction error and trigger the anomaly flag. This makes the system robust to novel emergency scenarios that were never explicitly labeled.

### 3.3.2 Data Pre-Processing: The Magnitude Vector Formula

Raw accelerometer data from a mobile device is orientation-dependent: the raw X, Y, and Z axis values change entirely if the user rotates their phone, places it face-down in a pocket, or attaches it to a bag strap. Training the LSTM on raw axis values would produce a model that is brittle and unreliable across different users and device placements.

The solution is to pre-process all raw accelerometer readings using the Total Acceleration Magnitude formula: A_mag = sqrt(Ax^2 + Ay^2 + Az^2). This mathematical transformation collapses the three-axis vector into a single scalar value representing the total magnitude of acceleration force acting on the device, regardless of orientation. A_mag is completely invariant to device orientation, enabling the model to generalize across all users and all device placements. All LSTM training and inference is performed on A_mag time series, not raw axis data.

### 3.3.3 Sliding Window Segmentation

The 50Hz continuous A_mag stream is segmented into fixed-length windows for batch processing by the LSTM. Each window is 3 seconds long, containing 150 data points. Windows overlap by 50% (1.5 seconds), meaning a new window is evaluated every 1.5 seconds. This overlap-based segmentation is architecturally critical: any sudden impact event lasting less than 1.5 seconds would be perfectly centered in at least one window, ensuring it is captured with full context on both sides rather than being split across a window boundary and losing its temporal signature.

### 3.3.4 The Twin Trigger Scenarios

| Scenario | Detection Logic |
|---|---|
| **Crash / Physical Impact Signature** | The A_mag vector exhibits a massive, instantaneous spike that far exceeds any normal activity threshold (high G-force event). The spike is followed by either chaotic, disorganized motion patterns (indicating post-crash disorientation or the vehicle tumbling) or an abrupt, complete cessation of motion (indicating the victim has been rendered immobile). The LSTM Reconstruction Error for this pattern spikes to many times the normal threshold, confirming anomaly. |

| | |
|---|---|
| **Immobility / Unconsciousness Signature** | The IMU data enters an absolute flatline state where the A_mag value stabilizes at or very near Earth's static gravitational constant of approximately 9.81 m/s^2. This indicates the device (and therefore the traveler) is completely stationary. Critically, this flatline must persist across multiple consecutive evaluation windows (establishing it is not a brief rest) AND must co-occur with a static GPS coordinate that places the traveler in a remote, isolated zone rather than a known rest area. The combination of prolonged IMU flatline plus remote static GPS position constitutes a confirmed unconsciousness alert. |

### 3.3.5 SNN-CAD: Sequential Nearest Neighbor Cumulative Anomaly Detection

The SNN-CAD algorithm operates as a secondary, spatial layer of anomaly detection that complements the LSTM's temporal analysis. While the LSTM analyzes the internal state of the traveler (their movement patterns), SNN-CAD analyzes the traveler's external positioning relative to known hazard geography.

SNN-CAD tracks the traveler's trajectory as a time-ordered sequence of GPS coordinates and computes the Hausdorff distance between the observed trajectory and historical safe route patterns for the same geographic area. The Hausdorff distance measures the maximum displacement between two trajectories, making it sensitive to situations where a traveler deviates significantly from established safe paths into unmapped or hazardous terrain. The algorithm achieves an Area Under the ROC Curve (AUC) of approximately 0.97, indicating exceptional discrimination between safe route deviations and genuinely dangerous geographic excursions. This provides predictive hazard alerts before an accident occurs, not just after.

### 3.3.6 Model Optimization for Mobile Inference

The trained TensorFlow/Keras LSTM Autoencoder model is exported to the ONNX (Open Neural Network Exchange) Runtime format for production deployment. ONNX Runtime inference is significantly faster than native TensorFlow inference for single-sample prediction tasks, which is the dominant use case here (one window evaluated every 1.5 seconds). The ONNX model is embedded in a dedicated FastAPI async worker pool, ensuring that ML inference does not block the main event loop handling WebSocket connections. Each worker independently evaluates incoming telemetry windows, and anomaly flags are emitted to the notification system asynchronously.

## 3.4 Haversine Distance Routing Algorithm

The Haversine Distance formula calculates the great-circle distance between two points on a sphere (the Earth) given their latitude and longitude coordinates. In TourSafe, this algorithm is used by the Node.js multi-agency dispatch microservice to determine which police station and which hospital are geographically closest to the traveler's distress location. This is not simply Euclidean distance — on Earth's curved surface, straight-line distance is inaccurate at scale, and Haversine correctly accounts for the spherical geometry. The dispatch system queries a database of pre-registered emergency resource locations (police stations, hospitals, EMS depots) and

ranks them by Haversine distance from the incident GPS coordinate, routing the automated e-FIR and dispatch notification to the optimal set of responders.

# 4. Blockchain Identity Layer — Decentralized Identity (DID)

## 4.1 The Self-Sovereign Identity Model

The Blockchain DID layer addresses the fundamental identity verification crisis at the heart of international travel emergencies. The core principle is Self-Sovereign Identity: the traveler, not any government or corporation, holds and controls their own identity credentials. These credentials are cryptographically secured, globally readable by authorized parties, and cannot be forged, altered, or denied by any single entity.

TourSafe implements the W3C Decentralized Identifier (DID) standard. Each traveler is issued a unique DID — a string identifier that is registered on the blockchain and resolves to a DID Document. The DID Document contains the traveler's public key, which is used to verify that encrypted credentials were indeed signed by the legitimate holder of that identity. The actual sensitive medical data is never stored on the public blockchain; it resides in a private, encrypted data vault, with only the public key and DID hash anchored on-chain for verification purposes.

## 4.2 Polygon PoS Network — Blockchain Infrastructure

The smart contracts are deployed on the Polygon Proof-of-Stake (PoS) network, specifically the Amoy Testnet (Chain ID: 80002) for development and testing, with mainnet deployment on Polygon PoS for production. The rationale for choosing Polygon over the Ethereum mainnet is decisive: Ethereum mainnet gas fees for contract interactions can range from a few dollars to hundreds of dollars per transaction during periods of network congestion, making it economically unviable for a system that may need to process thousands of identity verifications daily. Polygon PoS offers sub-second block finality and gas fees that are fractions of a cent, enabling the system to operate at scale without prohibitive costs. Polygon is also fully EVM-compatible, meaning all Ethereum tooling (ethers.js, Hardhat, MetaMask) works natively.

## 4.3 Smart Contract Architecture

### 4.3.1 The Identity Resolution Contract

The core smart contract, written in Solidity, is the Identity Resolution Contract. This contract maintains a mapping between a traveler's TourSafe account identifier and their on-chain DID record. The DID record stored on-chain contains the traveler's public key hash, the IPFS content address (CID) pointing to their encrypted data vault, and their registration timestamp. The contract exposes functions for DID registration (called once during traveler onboarding), DID resolution (called by first responders to retrieve the public key and vault address), and emergency access grant (an event emitted when an authenticated authority requests emergency decryption privileges).

### 4.3.2 Asymmetric Key Encryption Architecture

During the TourSafe onboarding process, the mobile application generates a public-private key pair using elliptic curve cryptography (specifically the secp256k1 curve, the same standard used by Bitcoin and Ethereum). The private key is stored exclusively in the device's secure hardware enclave using Expo SecureStore — it never leaves the device. The public key is submitted to the Identity Resolution Contract and stored on-chain.

The traveler's medical data vault is encrypted using the public key, meaning only the corresponding private key can decrypt it. This data vault contains: blood type, known allergies and medication sensitivities, chronic medical conditions, emergency contact details (name, phone, relationship), home country emergency service numbers, and travel insurance policy identifiers. The encrypted vault is stored on IPFS (InterPlanetary File System) for decentralized, censorship-resistant availability, with the IPFS CID anchored in the on-chain DID record.

### 4.3.3 Emergency Cryptographic Access — The QR Code Protocol

Each traveler's mobile application displays a dynamic QR code that encodes the traveler's DID identifier and a signed verification token. When a first responder scans this QR code using an authorized government agency device, the B2G dashboard performs the following sequence: it resolves the DID by querying the Identity Resolution Contract to retrieve the traveler's public key and IPFS vault address; it fetches the encrypted vault from IPFS; it uses an Emergency Cryptographic Access Key (a key held by the government agency, pre-registered in the smart contract as an authorized decryption delegate) to decrypt the vault; and it presents the traveler's critical medical information on the responder's screen within seconds.

**Zero-Knowledge Privacy Design**

Emergency Cryptographic Access Keys are granted to authority accounts strictly when an AI-confirmed emergency payload has been fired for that specific traveler. The access grant is time-limited and logged as an immutable transaction on the blockchain, creating an auditable trail of every identity access event. Outside of a confirmed emergency event, no authority can access the traveler's medical data — even if they possess a registered agency key. This design ensures medical data is never permanently exposed to government databases while still enabling instant access during the exact moments it is needed.

## 4.4 Hardhat Development & Deployment Pipeline

All Solidity smart contract development, compilation, testing, and deployment is managed through the Hardhat development environment. The Hardhat pipeline includes: TypeScript-based unit tests for every contract function using the Chai assertion library; a local Hardhat node for rapid development iteration; deployment scripts that target the Polygon Amoy Testnet for integration testing; and a Hardhat Verify plugin integration for publishing verified contract source code to the Polygonscan block explorer, enabling transparent public audit of the contract logic.

# 5. B2G Authority Dashboard & e-FIR Engine

## 5.1 Dashboard Architecture Overview

The Business-to-Government (B2G) Command Dashboard is the operational interface through which law enforcement, emergency medical services, and tourism safety officers monitor traveler safety in real time and coordinate emergency response. It is built on the MERN stack (MongoDB, Express.js, React.js, Node.js) with Mapbox GL JS for geospatial visualization.

The dashboard architecture is a microservices design: the React.js frontend is a single-page application that communicates with the backend via REST APIs and Socket.io event channels. The Node.js/Express.js backend exposes dedicated microservices for traveler telemetry queries, incident management, e-FIR generation and dispatch, and agency authentication. Each microservice can be deployed and scaled independently, enabling the system to handle geographic spikes in emergency volume without affecting unrelated dashboard functions.

## 5.2 Real-Time Map Visualization with Mapbox GL JS

Mapbox GL JS is a WebGL-based mapping engine that renders the live geographic intelligence layer of the dashboard. The Mapbox canvas displays: real-time GPS dots for every currently tracked traveler in the authority's jurisdiction, color-coded by their current safety status (green for normal, amber for geo-fence alert, red for confirmed emergency); live incident heatmap overlays aggregating historical anomaly data to reveal high-risk geographic zones over time; active geo-fence boundary polygons visualizing the virtual perimeters currently in effect; and the real-time trajectories of travelers who have triggered active anomaly alerts, allowing authorities to visually track the victim's last known movement direction.

## 5.3 Geo-Fencing System

The geo-fencing subsystem maintains a library of virtual perimeter polygons defining hazardous geographic zones — avalanche-prone mountain passes, flood-risk river valleys, restricted wildlife conservation areas, regions with historically elevated accident rates. These polygons are stored as GeoJSON objects in MongoDB and pushed to traveler mobile applications as part of the session initialization payload.

When a traveler's GPS coordinate crosses a geo-fence boundary — computed client-side using Turf.js for offline capability, and confirmed server-side — two simultaneous actions occur: the traveler's application displays an immediate push notification warning them of the hazardous zone entry; and the B2G dashboard receives a Socket.io event updating the traveler's status indicator to amber and logging the geo-fence breach to the incident timeline. If the traveler remains in the hazardous zone for more than a configurable duration (defaulting to 10 minutes) without a confirmed safe exit, the system escalates the alert status and notifies the duty officer.

## 5.4 Automated e-FIR Generation Engine

The Automated Electronic First Information Report (e-FIR) engine is a dedicated backend microservice that activates immediately upon AI anomaly confirmation. The e-FIR system is designed to eliminate the most critical administrative bottleneck in emergency response: the time required for a human dispatcher to gather information, fill out paperwork, and manually contact the relevant authorities.

Upon receiving a confirmed anomaly payload from the ML engine, the e-FIR microservice performs the following operations in parallel: it retrieves the traveler's decrypted DID credentials (medical data, identity, emergency contacts); it compiles the LSTM anomaly log including the reconstruction error trace and the time series of A_mag values surrounding the event; it packages the traveler's last known GPS coordinates with a timestamp; it formats all of this information into a structured JSON payload that conforms to a standardized machine-readable incident report schema; it generates a human-readable PDF version of the same report for jurisdictions that require paper-format records; and it dispatches both formats simultaneously to the API endpoints of the nearest jurisdictional police station node and the nearest hospital emergency department node, as determined by the Haversine distance calculation.

| e-FIR Field | Source |
|---|---|
| **Incident ID** | Auto-generated UUID by e-FIR microservice |
| **Incident Timestamp** | LSTM anomaly trigger timestamp from FastAPI backend |
| **Victim Identity** | Decrypted from Blockchain DID vault via Emergency Cryptographic Access Key |
| **Blood Type & Allergies** | Decrypted from Blockchain DID vault (medical sub-record) |
| **Emergency Contacts** | Decrypted from Blockchain DID vault (contact sub-record) |
| **Incident GPS Coordinates** | Last confirmed GPS packet from traveler telemetry stream |
| **Anomaly Classification** | LSTM trigger type: CRASH or IMMOBILITY/UNCONSCIOUSNESS |
| **LSTM Reconstruction Error** | Full error trace from LSTM Autoencoder evaluation window |
| **Dispatch Target — Police** | Nearest registered police node, determined by Haversine distance |
| **Dispatch Target — Hospital** | Nearest registered hospital emergency node, Haversine distance |
| **Geo-fence Status** | Active geo-fence violations at time of incident, if any |
| **Offline Data Flag** | Boolean indicating if any telemetry was reconstructed from offline queue |

# 6. Complete Technology Stack Reference

| Category | Technology | Role & Justification |
|---|---|---|
| Mobile (Client) | React Native CLI (Bare) + TypeScript | Cross-platform native iOS/Android from a single codebase, built from scratch using bare React Native CLI for unrestricted hardware access. Compile-time type safety for life-critical data structures. |
| Mobile (State) | Redux Toolkit + TanStack Query | Global state management (safety status, offline queue) and server-state synchronization. |
| Mobile (Maps) | Google Maps API + Turf.js | Map rendering and offline client-side geo-fence computation against cached GeoJSON boundaries. |
| Mobile (Offline) | SQLite + AsyncStorage | AES-256-encrypted offline telemetry queue for zero-connectivity resilience. |
| Mobile (Identity) | ethers.js + Expo SecureStore | On-device DID wallet, key generation, and secure hardware-backed private key storage. |
| Backend (Core) | FastAPI (Python, ASGI) | High-concurrency async WebSocket server for 50Hz telemetry ingestion from thousands of devices. |
| Backend (Cache) | Redis | Sub-millisecond in-memory cache for live GPS coordinates, driving real-time dashboard rendering. |
| Backend (DB) | MongoDB | Persistent schema-flexible storage for traveler profiles, incident logs, and e-FIR archives. |
| Backend (Realtime) | Socket.io | Persistent bidirectional WebSockets for instant incident push to authority dashboard. |
| ML Engine | TensorFlow + Keras + ONNX Runtime | LSTM Autoencoder training (TF/Keras) and production-optimized inference (ONNX Runtime). |
| ML Algorithms | LSTM Autoencoder + SNN-CAD + Haversine | Temporal anomaly detection, spatial trajectory hazard detection, optimal dispatch routing. |
| Blockchain | Polygon PoS (Amoy Testnet -> Mainnet) | DID anchoring with sub-cent gas fees and sub-second finality. EVM-compatible. |

| | | |
|---|---|---|
| Smart Contracts | Solidity + Hardhat | Identity Resolution Contract: DID registration, resolution, and emergency access grants. |
| Cryptography | secp256k1 Asymmetric Keys + AES-256 | Asymmetric key pairs for DID vault encryption; AES-256 for offline telemetry buffer encryption. |
| Identity Standard | W3C Decentralized Identifiers (DIDs) | Self-Sovereign Identity standard enabling first responder QR-code-based identity resolution. |
| Dashboard (Frontend) | React.js + Mapbox GL JS | WebGL-powered real-time heatmap and traveler tracking visualization for authority operators. |
| Dashboard (Backend) | Node.js + Express.js | e-FIR generation microservice and jurisdictional dispatch routing API. |
| Development Tooling | VS Code + AnthropicAI + GitHub Copilot + Android Studio | Primary IDE: VS Code with AnthropicAI extension and GitHub Copilot for AI-assisted development across all modules. Android Studio used for AVD (Android Virtual Device) emulator setup, device debugging, and APK build validation during Android-specific phases. |
| Containerization | Docker + Docker Compose | Reproducible development and deployment environment for all backend services. |
| IoT (MVP Prototype) | ESP32 DevKit V1 / Raspberry Pi Pico W | Edge compute for physical sensor validation without relying solely on mobile app simulation. |
| Sensors (MVP) | MPU6050 IMU + NEO-6M GPS Module | Hardware sensor array for physical crash simulation and GPS validation in prototype testing. |
| Privacy | Zero-Knowledge Proofs + Ephemeral Data | ZKP for selective credential disclosure; telemetry purged post-trip, only anonymized logs retained. |

# 7. Implementation Plan — 4-Sprint Agile Roadmap

## 7.1 Sprint Overview

| Sprint | Focus Area & Deliverables |
|---|---|
| Sprint 1 | Core Infrastructure & Scaffolding — Docker environment, FastAPI backbone, MERN dashboard scaffolding, React Native app skeleton. |
| Sprint 2 | Client Telemetry & Offline Caching — Background GPS and IMU sensor polling, AES-256 offline buffer, Turf.js geo-fence client implementation. |
| Sprint 3 | ML Execution & Authority Communication — LSTM training and ONNX deployment, SNN-CAD integration, Socket.io real-time streaming to dashboard. |
| Sprint 4 | Blockchain DID & e-FIR Automation — Solidity contract deployment, asymmetric key vault, QR code emergency access, automated e-FIR dispatch. |

## 7.2 Sprint 1 — Core Infrastructure & Scaffolding

### Phase 1A: Containerized Backend Environment

The first development phase establishes the foundational infrastructure. Docker Compose is used to define and orchestrate the complete backend environment as a multi-container application. The initial Docker Compose configuration spins up three containers: the FastAPI application server, a MongoDB instance with a pre-seeded schema for traveler profiles and incident collections, and a Redis instance configured for GPS coordinate caching. All inter-service networking, port mappings, and environment variable injection are defined in the Docker Compose file, ensuring the entire backend can be reproduced on any developer machine with a single command.

### Phase 1B: FastAPI API Backbone

The FastAPI application is initialized with a clean modular directory structure. Pydantic data models are defined for every data type that will flow through the system: the incoming telemetry packet (traveler_id, timestamp, gps_lat, gps_lng, accel_x, accel_y, accel_z, amag), the processed anomaly event (traveler_id, event_type, reconstruction_error, gps_coordinate, timestamp), and the incident dispatch payload (all anomaly fields plus decrypted identity fields). The primary asynchronous WebSocket handler for the /ws/telemetry endpoint is constructed and tested for basic connectivity. REST endpoints for traveler registration, geo-fence boundary retrieval, and incident history queries are scaffolded.

### Phase 1C: MERN Dashboard Scaffolding

The React.js dashboard application is initialized. The Mapbox GL JS canvas is embedded and configured with the appropriate API key and initial geographic viewport. A basic routing structure is established for the dashboard views: the Live Map view (primary operational view), the Incident Feed view (chronological list of recent anomaly events), the Traveler Directory view (searchable registry of registered travelers in jurisdiction), and the e-FIR Archive view. Authentication middleware for authority agency login is implemented using JWT tokens. The Node.js/Express.js backend for the dashboard is initialized with a Socket.io server that will later receive anomaly broadcasts from FastAPI.

### Phase 1D: React Native App Skeleton

The React Native TypeScript application is initialized using the React Native CLI (bare workflow), ensuring unrestricted native module access for hardware-intensive operations. The development environment is configured with Visual Studio Code as the primary IDE, equipped with the AnthropicAI extension and GitHub Copilot for AI-assisted code generation, IntelliSense, and refactoring throughout the project. Android Studio is installed and configured with at least one Android Virtual Device (AVD) — targeting API Level 33 or higher — to serve as the emulator environment for all Android-specific development, testing, and build validation phases. The core navigation structure is implemented using React Navigation, establishing the primary screens: the Onboarding Flow (DID registration, medical data entry, emergency contact setup), the Home/Dashboard screen (showing current safety status, active geo-fence alerts, and connection indicator), the Live Map screen (showing the traveler's own position and nearby safe zones), and the Emergency Override screen (manual SOS button for conscious users who wish to manually trigger an alert). Permission requests for background location and sensor access are implemented at this stage.

## 7.3 Sprint 2 — Client Telemetry & Offline Buffering

### Phase 2A: Background Sensor Pipeline

The Expo Sensors API and Expo Location API are integrated into the React Native application. The background location service is configured to poll GPS at 1Hz and transmit updates to the offline buffer or WebSocket channel. The IMU accelerometer listener is configured to poll at 50Hz. The sliding window accumulator is implemented in the application's Redux middleware: as samples arrive from the sensor listener, they are added to a rolling ring buffer. Every 1.5 seconds (50% overlap of the 3-second window), the ring buffer is read, the A_mag magnitude is computed for each sample in the window, and the resulting normalized window is packaged as a telemetry payload ready for transmission.

### Phase 2B: SQLite Offline Buffer with AES-256 Encryption

The SQLite database is initialized within the React Native application. A TelemetryQueue table is created with columns for: a unique row identifier, the serialized and encrypted telemetry payload, the original capture timestamp, and a synchronization status flag (PENDING, IN_FLIGHT, COMMITTED, FAILED). The network interceptor middleware is implemented: before every WebSocket transmission, a lightweight connectivity ping is attempted. On ping timeout, the payload is serialized to JSON, encrypted using AES-256-CBC with a session-derived key, and inserted into the TelemetryQueue as a PENDING row. The background autocommit job is

implemented as a React Native background task that wakes every 30 seconds, checks connectivity, and if a connection is available, processes all PENDING rows in the queue.

### Phase 2C: Client-Side Geo-Fencing

The geo-fence boundary data (stored as GeoJSON FeatureCollection polygons in MongoDB) is fetched from the FastAPI backend during session initialization and cached locally in the application's Redux store and SQLite database for offline access. The Turf.js booleanPointInPolygon function is used to evaluate the traveler's current GPS coordinate against every cached geo-fence polygon on each 1Hz GPS update. If the traveler enters a geo-fence, a local push notification is displayed immediately (without requiring a server round-trip), and a geo-fence breach event is logged and transmitted to the backend. This client-side evaluation ensures geo-fence alerts function even with zero network connectivity.

## 7.4 Sprint 3 — ML Execution & Authority Communication

### Phase 3A: LSTM Model Training & ONNX Export

The LSTM Autoencoder is designed with a Sequence-to-Sequence architecture using TensorFlow and Keras. The encoder consists of two stacked LSTM layers that progressively compress the 150-timestep input sequence into a compact latent vector. The decoder consists of a RepeatVector layer to expand the latent vector back to sequence length, followed by two LSTM layers that reconstruct the original sequence. The model is trained on a curated dataset of A_mag time series collected from normal human activities. Training is performed with a Mean Squared Error loss function and the Adam optimizer. The trained model is exported to ONNX format using the tf2onnx library and validated to confirm output parity between the TensorFlow and ONNX versions. The ONNX model is deployed to the FastAPI server and loaded into an async worker pool using the ONNX Runtime Python library.

### Phase 3B: Reconstruction Error Threshold Calibration

The anomaly detection threshold for the LSTM Reconstruction Error is calibrated using a holdout validation dataset of normal activity data. The threshold is set at the 99.5th percentile of reconstruction errors observed on normal data — meaning approximately 0.5% of normal windows will trigger a false positive. Given the 1.5-second window evaluation interval, this translates to approximately one false positive per roughly 5 minutes of continuous monitoring, which is acceptable for a safety application where false positives result in a confirmation query rather than an immediate emergency response. A two-stage confirmation protocol is implemented: a first-stage flag when the reconstruction error crosses the calibrated threshold, followed by a confirmed anomaly classification only when two consecutive overlapping windows both exceed the threshold, eliminating single-window noise spikes.

### Phase 3C: SNN-CAD Integration & WebSocket Streaming

The SNN-CAD algorithm is implemented as a Python module in the FastAPI backend. For each traveler, the algorithm maintains a running trajectory history and evaluates the Hausdorff distance between the recent GPS trajectory and the nearest historical safe route pattern from the geographic database. When the distance exceeds the calibrated hazard threshold, a trajectory

anomaly event is generated and included in the traveler's status record. The Socket.io event emitter is integrated into the FastAPI anomaly detection pipeline. When an LSTM anomaly or SNN-CAD trajectory anomaly is confirmed, the event emitter broadcasts a structured incident packet to the Socket.io server on the MERN dashboard backend, which immediately pushes the event to all connected authority dashboard clients subscribed to the relevant geographic zone.

## 7.5 Sprint 4 — Blockchain DID & e-FIR Automation

### Phase 4A: Smart Contract Development & Deployment

The Identity Resolution Contract is written in Solidity. The contract defines the DID record structure and a mapping from traveler account addresses to their DID records. Key functions include: registerDID (called once during onboarding, stores the traveler's public key hash and IPFS vault CID on-chain), resolveDID (called by first responders to retrieve public key and vault address), grantEmergencyAccess (called by the TourSafe backend when an AI emergency is confirmed, emitting an on-chain event that authorizes temporary decryption for a specific agency), and revokeEmergencyAccess (called after the emergency is resolved, closing the temporary access window). The contract is compiled and tested using Hardhat with TypeScript tests, then deployed to the Polygon Amoy Testnet.

### Phase 4B: Mobile DID Onboarding & Vault Encryption

The React Native onboarding flow is extended to include the DID registration process. Upon first app launch, the application uses ethers.js to generate a secp256k1 key pair. The private key is stored in Expo SecureStore (hardware-backed secure enclave). The traveler is guided through entering their medical information and emergency contacts. This data is serialized to JSON, encrypted using the generated public key, and uploaded to IPFS via a Pinata or Web3.Storage API call, returning an IPFS CID. The CID and public key are then submitted to the Identity Resolution Contract's registerDID function via a signed transaction, completing the on-chain DID registration. The traveler's TourSafe QR code is generated from their DID identifier and a signed verification token.

### Phase 4C: Emergency Decryption & e-FIR Microservice

The e-FIR microservice is implemented as a Node.js/Express.js service. Upon receiving a confirmed anomaly payload from the FastAPI ML engine, the microservice initiates the emergency decryption sequence: it calls the smart contract's grantEmergencyAccess function using the government agency's registered key, then fetches the encrypted vault from IPFS using the traveler's CID. The vault is decrypted using the agency's Emergency Cryptographic Access Key. The decrypted identity data is merged with the anomaly logs, GPS coordinates, and LSTM reconstruction error traces. The assembled report is formatted as both a JSON payload and a PDF document. Both formats are dispatched to the pre-registered API endpoints of the nearest jurisdictional police station and hospital emergency nodes, determined by Haversine distance calculation. The complete e-FIR is archived in MongoDB with a dispatch status log.

# 8. Privacy, Ethics & Regulatory Compliance

## 8.1 Zero-Knowledge Privacy Architecture

TourSafe's privacy model is built on two foundational principles: Zero-Knowledge Proofs (ZKP) for selective credential disclosure and Ephemeral Data Retention for telemetry data. ZKP enables a traveler to prove specific facts about their identity (e.g., that they are registered with TourSafe and have a valid medical profile) to a verifier without revealing the underlying data. In practice, this means a first responder can confirm that a QR code is legitimate and that an emergency access request is valid without the system needing to transmit raw personal data until the decryption is explicitly authorized by the emergency event trigger.

## 8.2 Ephemeral Data Retention Policy

All raw telemetry data — GPS coordinates, IMU readings, trajectory logs — is automatically purged from the TourSafe servers at the conclusion of each trip, defined as 24 hours after the traveler's registered return date or when the traveler manually closes their active session. What is retained long-term is only anonymized, aggregated incident data for the purpose of improving the safety heatmap models and calibrating the LSTM thresholds. No personally identifiable information is associated with long-term retained data. This policy is enforced through automated scheduled database jobs that run nightly and is documented in the traveler's consent agreement at onboarding.

## 8.3 On-Chain Audit Trail & Access Governance

Every emergency access event — every instance where a government agency key is used to decrypt a traveler's medical vault — is recorded as an immutable transaction on the Polygon blockchain. This creates a permanent, tamper-proof audit trail that can be reviewed by the traveler, by TourSafe's compliance team, and by regulatory authorities. The smart contract's access control model ensures that even TourSafe's own administrative accounts cannot decrypt traveler data outside of an AI-confirmed emergency event, providing a technical enforcement mechanism rather than relying solely on policy controls.

## 8.4 Data Localization & Cross-Border Compliance

Cross-border data sharing restrictions pose a significant compliance challenge for a global travel safety platform. TourSafe addresses this through a jurisdiction-aware data architecture: traveler data is processed and stored in cloud regions that match the traveler's destination country, ensuring compliance with local data residency laws (GDPR in Europe, PDPA in India, CCPA in California, etc.). The Blockchain DID layer further mitigates cross-border compliance complexity by ensuring that the identity verification process does not require transmitting raw personal data across borders — the first responder's device locally decrypts the vault using a key grant from the smart contract, and the decrypted data remains on the responder's device rather than being transmitted through cross-border servers.

# 9. Hardware MVP Prototype & Physical Validation

## 9.1 IoT Edge Compute Architecture

For showcase and validation purposes, TourSafe's AI models and sensor pipeline are prototyped on a hardware IoT platform that physically demonstrates the system's capabilities without relying solely on mobile application simulation. This hardware prototype provides a compelling, tangible demonstration of the system's core functionality and validates the sensor-to-anomaly detection pipeline in a controlled physical environment.

| Component | Role in Prototype |
|---|---|
| **ESP32 DevKit V1 or Raspberry Pi Pico W** | Primary edge compute unit. Both options feature built-in Wi-Fi and Bluetooth for wireless telemetry transmission, low power consumption for portable battery operation, and sufficient processing capacity to run sensor polling at 50Hz and compute the A_mag magnitude formula locally before transmission. |
| **MPU6050 IMU Module** | The primary sensor. The MPU6050 is a 6-axis IMU combining a 3-axis accelerometer and a 3-axis gyroscope on a single chip, communicating via I2C protocol. It generates the raw Ax, Ay, Az acceleration values that form the input to the LSTM anomaly detection pipeline. |
| **NEO-6M GPS Module** | Provides real-time latitude/longitude coordinates via UART serial communication. Supplies the geographic positioning data required for immobility detection (confirming static GPS position) and for Haversine-based dispatch routing in the prototype demo. |
| **Momentary Push-Button** | Simulates the manual SOS override functionality — a secondary trigger mechanism for conscious travelers who wish to manually initiate an emergency alert alongside the autonomous AI detection system. |
| **18650 Li-ion Battery + TP4056 Module** | Provides portable, rechargeable power for fully untethered field demonstration. The TP4056 charging module enables USB-C recharging without removing the battery from the prototype enclosure. |
| **5V Active Buzzer + RGB LED** | Physical feedback peripherals. The RGB LED provides visual status indication: green for normal tracking, amber for geo-fence breach, red for confirmed anomaly. The buzzer provides an audible alert when an SOS is triggered or a geo-fence is breached, making the system's status unambiguous to demo observers. |

## 9.2 Physical Testing Protocol

The hardware prototype validation protocol is designed to demonstrate three core system behaviors in a live setting. For crash detection validation, the MPU6050 sensor array is manually shaken with sharp, high-G-force impulses to simulate a vehicular crash event. The A_mag spike generated by this physical action is transmitted to the LSTM inference engine, and the expected

outcome is a confirmed anomaly event that triggers an e-FIR generation on the dashboard within seconds. For offline buffer validation, the ESP32's Wi-Fi connection is disabled mid-session to simulate zero-coverage conditions. Telemetry data accumulates in the local buffer, and when Wi-Fi is re-enabled, the autocommit flush is observed restoring the complete data record to the server. For geo-fence breach validation, the GPS module's position is spoofed to simulate a traveler entering a pre-configured hazard zone boundary, and the expected outcome is an immediate amber alert on the dashboard with a corresponding push notification displayed on the demo mobile device.

# 10. Business Model & Financial Architecture

## 10.1 Primary Revenue Stream: B2G Zone Licensing

TourSafe's primary commercial model is a Business-to-Government (B2G) annual licensing arrangement. Local government bodies, state tourism departments, and municipal authorities pay an annual zone licensing fee of ₹20,000,000 (₹2 Crore) per designated Safe Zone. This licensing fee grants the authority access to the full B2G Command Dashboard for their jurisdiction, including real-time traveler monitoring, incident heatmaps, automated e-FIR generation, and access to TourSafe's aggregated safety intelligence reports. The B2G model is preferred because government procurement provides stable, multi-year contract revenue and eliminates the need for direct-to-consumer marketing for the foundational platform.

## 10.2 Secondary Revenue Stream: B2B Per-Trip Commission

TourSafe's secondary revenue stream is a Business-to-Business commission model targeting travel agencies, tour operators, hotel groups, and adventure tourism companies. These partners pay a per-trip fee of ₹499 per traveler per registered trip. In exchange, their customers receive TourSafe protection during the trip duration, and the partner business receives access to a lightweight safety dashboard for their customer portfolio and a 'TourSafe Certified' marketing designation for their offerings. This model scales directly with tourism volume and requires minimal incremental infrastructure cost per additional traveler.

## 10.3 Key Performance Indicators

| KPI | Definition & Target |
|---|---|
| **Mean Time to Respond (MTTR)** | The elapsed time from AI anomaly confirmation to first responder acknowledgment of the e-FIR. Target: under 5 minutes, representing a 70%+ reduction from the current 20-30 minute baseline. |
| **False Positive Rate** | The percentage of LSTM anomaly flags that do not correspond to a genuine emergency event. Target: less than 2% of flags result in unnecessary emergency dispatch after the two-stage confirmation protocol. |
| **Active User Monitoring Coverage** | The number of travelers whose telemetry is being actively processed by the TourSafe system at any given moment. Target: 5,000+ travelers across initial rollout zones within 12 months of launch. |
| **Offline Data Recovery Rate** | The percentage of telemetry data captured during offline periods that is successfully recovered and delivered via the autocommit mechanism. Target: greater than 99.9%. |
| **DID Resolution Latency** | The time from first responder QR code scan to successful display of decrypted medical information. Target: under 15 seconds. |

| | |
|---|---|
| **e-FIR Dispatch Latency** | The time from AI anomaly confirmation to successful delivery of the e-FIR to police and hospital nodes. Target: under 60 seconds. |

## 10.4 SDG Alignment

TourSafe's implementation directly contributes to three United Nations Sustainable Development Goals. SDG 8 (Decent Work and Economic Growth) is served by protecting the tourism economy — a major employment driver — from safety incidents that deter travelers and suppress destination revenue. SDG 11 (Sustainable Cities and Communities) is served by making tourist destinations safer and more resilient to emergency incidents through proactive monitoring and faster response infrastructure. SDG 16 (Peace, Justice and Strong Institutions) is served by the automated e-FIR system, which strengthens law enforcement institutions' capacity to respond to and document tourist safety incidents with greater speed, accuracy, and accountability.

# 11. System Integration Map & Module Dependencies

## 11.1 Module Dependency Overview

Understanding the dependency relationships between TourSafe's modules is critical for correct implementation sequencing and for diagnosing integration failures. The following table maps each module to its upstream dependencies (what it requires to function) and its downstream consumers (what depends on it).

| Module | Depends On | Consumed By |
|---|---|---|
| React Native Mobile App | FastAPI WebSocket, Polygon RPC, Google Maps API, IPFS | Traveler (end user interface) |
| Sensor Collection Pipeline | Expo Sensors API, Expo Location API, Redux Toolkit | SQLite Offline Buffer, WebSocket Transmitter |
| SQLite Offline Buffer | AES-256 symmetric key, TelemetryQueue schema | Autocommit Job, WebSocket Transmitter |
| FastAPI WebSocket Handler | Pydantic models, asyncio event loop, ONNX Runtime | Redis GPS Cache, MongoDB Archive, Anomaly Engine |
| LSTM ONNX Inference Engine | ONNX model file, A_mag pre-processing utility | Anomaly Event Emitter, e-FIR Microservice |
| SNN-CAD Algorithm | Historical safe route GeoJSON data, Hausdorff distance library | Anomaly Event Emitter, Dashboard Geo-fence Layer |
| Haversine Dispatch Router | Emergency resource location database (police, hospitals) | e-FIR Microservice dispatch targets |
| Redis GPS Cache | FastAPI write operations, Redis instance | MERN Dashboard live map rendering |
| MongoDB | FastAPI and Node.js write operations | Dashboard queries, e-FIR archive, heatmap analytics |
| Socket.io Event Emitter | FastAPI anomaly confirmation, Socket.io server on dashboard | MERN Dashboard real-time incident feed |
| Polygon Smart Contract | Hardhat deployment, Polygon Amoy/Mainnet RPC | Mobile DID registration, e-FIR emergency access grant |
| IPFS Encrypted Vault | Public key encryption, Pinata/Web3.Storage API | e-FIR microservice decryption, first responder QR access |
| e-FIR Microservice | LSTM anomaly payload, Decrypted DID vault, Haversine router | Jurisdictional police API, hospital emergency API |
| MERN Dashboard | Socket.io stream, REST APIs, Mapbox GL JS, MongoDB queries | Authority operators (police, EMS, tourism safety officers) |

| | | |
|---|---|---|
| Mapbox GL JS | Redis live GPS data, GeoJSON geo-fence boundaries, incident heatmap data | Dashboard Live Map view |

# 12. Testing Strategy & Quality Assurance

## 12.1 Unit Testing

Every discrete functional module in TourSafe is covered by unit tests before integration. The A_mag magnitude calculation function is tested against known input vectors and expected output values to confirm mathematical correctness. The sliding window segmentation function is tested with edge cases including window boundaries, exactly-full buffers, and partial windows at stream start and end. The Haversine distance function is tested against known geographic coordinate pairs with independently verified distances. The AES-256 encryption and decryption functions are tested for round-trip correctness and for correct failure behavior on tampered ciphertexts. The Solidity smart contract functions are tested using Hardhat's TypeScript test suite, covering all state transitions, access control rules, and event emissions.

## 12.2 Integration Testing

Integration tests validate the complete data flow through connected module pairs. The telemetry pipeline integration test sends a synthetic 50Hz sensor stream from a test client to the FastAPI WebSocket endpoint and verifies that: the correct A_mag values are computed server-side; the ONNX model inference completes within the latency budget (under 100 milliseconds per window); and anomaly events are correctly emitted to the Socket.io test subscriber. The offline buffer integration test disconnects the test device's network mid-stream, verifies that telemetry is queued in SQLite with correct encryption, reconnects the network, and verifies that all queued data is correctly flushed and received by the server. The DID integration test runs the complete onboarding-to-emergency-decryption flow against the Polygon Amoy Testnet.

## 12.3 Load & Stress Testing

The FastAPI backend is load-tested using Locust to simulate the expected production load of simultaneous traveler connections. Test scenarios include: 1,000 concurrent WebSocket connections each sending 50Hz telemetry (50,000 data points per second total ingestion rate); 5,000 concurrent connections with 10% of them simultaneously triggering anomaly events (500 simultaneous e-FIR generation requests); and a complete system stress test that maintains maximum concurrent load while also executing Blockchain DID resolutions and IPFS vault fetches. The acceptance criterion is that the 99th-percentile response latency for anomaly detection (from telemetry receipt to Socket.io dashboard push) remains under 500 milliseconds at peak load.

## 12.4 Hardware Prototype Validation

The IoT hardware prototype undergoes three validation tests: physical crash simulation (the MPU6050 is subjected to controlled high-G-force impulses and the LSTM response is recorded), offline resilience testing (the Wi-Fi connection is disabled for extended periods and the buffer recovery is validated), and geo-fence boundary testing (the GPS module position is programmatically moved across a pre-configured geo-fence boundary to confirm client-side and server-side detection concordance).

# 13. Deployment Architecture & DevOps

## 13.1 Containerization Strategy

The entire TourSafe backend is containerized using Docker and orchestrated with Docker Compose for development environments and Kubernetes for production. Each backend service — FastAPI, MongoDB, Redis, the e-FIR Node.js microservice, and the Socket.io relay — runs in its own isolated container with defined resource limits and health check endpoints. This containerization ensures environment parity between development, staging, and production, eliminating the classic 'works on my machine' failure mode that is particularly dangerous for a life-critical system.

## 13.2 Cloud Deployment Targets

The TourSafe production infrastructure is designed for deployment on a major cloud provider (AWS, GCP, or Azure) using managed Kubernetes (EKS, GKE, or AKS). The FastAPI backend is deployed as a horizontally scalable deployment, with auto-scaling triggered by WebSocket connection count and telemetry ingestion throughput. MongoDB is deployed as a managed Atlas cluster with geographic replication to ensure data residency compliance. Redis is deployed as a managed ElastiCache or Memorystore instance with high-availability configuration. The MERN dashboard frontend is deployed as a static build to a CDN for global low-latency access by authority users.

## 13.3 CI/CD Pipeline

A continuous integration and continuous deployment pipeline is configured using GitHub Actions. Every pull request triggers the full unit test suite, integration tests against a staging environment, and a Hardhat test run against the Polygon Amoy Testnet. Successful merges to the main branch trigger automated Docker image builds and deployment to the staging Kubernetes cluster. Production deployments require manual approval from a designated release manager, ensuring no untested changes reach the live system protecting travelers.

# 14. Risk Register & Mitigation Strategies

| Risk | Likelihood / Impact | Mitigation Strategy |
|---|---|---|
| LSTM False Positive Rate exceeds acceptable threshold, causing unnecessary emergency dispatch | Medium / High | Two-stage confirmation protocol (two consecutive window breaches required). Human-in-the-loop confirmation step on dashboard before full e-FIR dispatch. Continuous threshold recalibration using production feedback data. |
| Network connectivity loss in remote areas prevents telemetry transmission | High / Medium | Primary mitigation: AES-256 SQLite offline buffer with autocommit. Secondary: Bluetooth mesh networking between nearby TourSafe users for peer-relay telemetry in zero-connectivity zones (Phase 2 feature). |
| Blockchain gas fee spikes on Polygon impacting DID transaction costs | Low / Medium | Transaction batching for non-emergency DID registrations. Emergency access transactions are given gas price priority. Budget allocation for gas costs with monthly review. |
| IPFS vault availability issues preventing emergency decryption | Low / High | Dual-pinning to both Pinata and Filecoin for redundancy. Encrypted vault backup stored in MongoDB with access gated by the same smart contract access control logic. |
| Smart contract vulnerability enabling unauthorized medical data access | Low / Critical | Mandatory third-party smart contract security audit before mainnet deployment. Formal verification of critical access control functions. Bug bounty program post-launch. |
| Battery drain on traveler's device due to continuous 50Hz sensor polling | High / Medium | Adaptive polling rate: 50Hz during active travel, reduced to 5Hz during stationary periods (detected by GPS stability). Foreground/background sensor management to comply with iOS and Android battery optimization guidelines. |
| Regulatory non-compliance with local health data laws (HIPAA, GDPR, PDPA) | Medium / High | Jurisdiction-aware data residency architecture. DID vault encryption ensures raw data is never transmitted in plaintext. Legal review of data handling procedures in each target deployment region. |
| First responders in target regions lacking the technical capability to use the QR scan system | Medium / High | Dedicated onboarding training program for registered law enforcement agencies. QR code fallback to spoken DID identifier for voice relay. Dashboard accessible via standard web browser without specialized software. |

# 15. Future Development Roadmap

## Phase 2 Enhancements (Post-MVP)

- Bluetooth Mesh Peer Relay Network: Enable TourSafe users in the same geographic area to form ad-hoc Bluetooth mesh networks that relay telemetry data from zero-connectivity zones through peers with partial connectivity, creating a self-healing coverage mesh in remote areas.
- Satellite Connectivity Integration: Integrate with Low-Earth Orbit satellite communication providers (Starlink, Iridium) for guaranteed connectivity backup in truly remote zones where even cellular infrastructure is absent.
- Wearable Device Extension: Develop a companion application for Apple Watch, Wear OS, and dedicated sports wearables that can collect biometric data (heart rate, blood oxygen saturation, skin conductance) to augment the IMU-based anomaly detection with physiological signals, significantly improving detection accuracy for medical emergencies like cardiac events.
- Computer Vision Incident Verification: Integrate an on-device computer vision model that activates upon LSTM anomaly confirmation to capture and analyze visual context from the device camera, providing corroborating evidence for the e-FIR and enabling visual damage assessment for vehicular incidents.
- Multilingual NLP Emergency Communication: Implement a real-time translation layer that allows first responders and victims to communicate through the TourSafe app across language barriers during the response window, using on-device NLP models to minimize latency.
- Predictive Risk Scoring: Develop a personalized risk score for each traveler based on their planned itinerary, historical incident data for their destination zones, current weather and environmental conditions, and their own health profile — providing proactive safety recommendations before incidents occur.

## Phase 3 Enhancements (Scale)

- Global Safe Zone Certification Network: Establish a formal TourSafe Certified Safe Zone accreditation program for destinations worldwide, with standardized safety metrics and public-facing certification badges that travel booking platforms can display.
- Insurance Industry API Integration: Create a standardized API that travel insurance providers can integrate with, enabling automatic incident reporting, claim initiation, and real-time policy validation during emergencies without the insured traveler needing to file claims manually.
- Cross-Border Emergency Protocol Standardization: Work with international organizations to standardize the e-FIR data schema as an international travel emergency incident reporting format, enabling seamless cross-border data exchange between national emergency systems.

# 16. Glossary of Key Technical Terms

| Term | Definition |
|---|---|
| **LSTM (Long Short-Term Memory)** | A type of recurrent neural network architecture specifically designed to learn long-range temporal dependencies in sequential data, making it ideal for analyzing time-series sensor data. |
| **Autoencoder** | A neural network architecture trained to compress input data into a compact representation and then reconstruct the original input. The quality of reconstruction is used as an anomaly signal. |
| **Reconstruction Error** | The quantitative difference (typically measured as Mean Squared Error) between the original input to an Autoencoder and its reconstructed output. High reconstruction error indicates an anomalous input. |
| **A_mag (Acceleration Magnitude)** | The scalar total magnitude of a three-axis acceleration vector, computed as sqrt(Ax^2 + Ay^2 + Az^2). Orientation-invariant and used as the primary LSTM input feature. |
| **SNN-CAD** | Sequential Nearest Neighbor Cumulative Anomaly Detection. A spatial trajectory analysis algorithm that uses Hausdorff distance to detect geographic deviations from safe historical routes. |
| **Hausdorff Distance** | A mathematical measure of the maximum distance between two sets of points (here, GPS trajectories), representing the worst-case spatial deviation between the observed and expected routes. |
| **Haversine Distance** | A formula for calculating the great-circle distance between two points on a sphere's surface given their latitude and longitude coordinates. Used for accurate GPS-based distance measurement. |
| **DID (Decentralized Identifier)** | A W3C standard for a globally unique, cryptographically verifiable identifier that is not dependent on any centralized registry, authority, or identity provider. |
| **Self-Sovereign Identity (SSI)** | An identity model in which the individual controls their own identity credentials without reliance on a central authority to issue, store, or validate them. |
| **Polygon PoS** | Polygon Proof-of-Stake is a Layer 2 blockchain scaling solution for Ethereum, offering fast transactions with sub-cent gas fees while maintaining EVM compatibility. |
| **Smart Contract** | Self-executing code stored on a blockchain that automatically enforces the terms of an agreement when predefined conditions are met, without requiring a trusted intermediary. |
| **ASGI** | Asynchronous Server Gateway Interface. A standard interface for Python web applications and frameworks to communicate with web servers in a way that natively supports asynchronous operations. |

| | |
|---|---|
| **AES-256** | Advanced Encryption Standard with a 256-bit key length. A symmetric block cipher considered cryptographically secure and used for encrypting offline telemetry data. |
| **e-FIR** | Electronic First Information Report. A digital, machine-readable version of the standardized incident report filed with law enforcement upon detection of a crime or emergency. |
| **Geo-fencing** | The use of virtual geographic boundary perimeters (defined as GeoJSON polygons) to trigger alerts or actions when a tracked entity enters or exits the defined area. |
| **AUC** | Area Under the ROC Curve. A performance metric for binary classifiers ranging from 0 to 1, where 1 represents perfect discrimination and 0.5 represents random chance. SNN-CAD achieves ~0.97. |
| **ZKP (Zero-Knowledge Proof)** | A cryptographic method by which one party can prove to another that they know a value or fact, without conveying any information beyond the fact of the claim's truth. |
| **IPFS** | InterPlanetary File System. A decentralized, peer-to-peer file storage protocol where content is addressed by its cryptographic hash rather than by a server location. |
| **WebSocket** | A communication protocol that provides full-duplex communication channels over a single TCP connection, enabling real-time bidirectional data exchange between client and server. |
| **ONNX Runtime** | Open Neural Network Exchange Runtime. A cross-platform, high-performance ML inference engine that executes models in the standardized ONNX format faster than native framework inference. |

# 17. References, Standards & Compliance Frameworks

## 17.1 Technical Standards Cited

- W3C Decentralized Identifiers (DIDs) v1.0 — https://www.w3.org/TR/did-core/
- W3C Verifiable Credentials Data Model v1.1 — https://www.w3.org/TR/vc-data-model/
- NIST FIPS 197 — Advanced Encryption Standard (AES-256)
- EIP-1056 — Ethereum Improvement Proposal for Lightweight Identity on Ethereum
- OpenAPI 3.0 Specification — FastAPI auto-generated API documentation standard
- ONNX IR v8 — Open Neural Network Exchange Intermediate Representation
- GeoJSON RFC 7946 — Geographic data format for geo-fence polygon definitions

## 17.2 Regulatory Frameworks

- India PDPA (Personal Data Protection Act) — Data localization and consent requirements for Indian traveler data
- EU GDPR (General Data Protection Regulation) — Cross-border data handling and right-to-erasure compliance for European travelers
- UN SDG Framework — Goals 8, 11, and 16 alignment documentation
- India IT Act 2000 and IT Amendment Act 2008 — e-FIR legal validity and digital evidence standards

## 17.3 Key Academic & Industry References


- Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.
- World Health Organization. (2023). International Travel and Health: Traveler fatality statistics.
- Polygon Technology. (2024). Polygon PoS Technical Documentation. polygon.technology.
- Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. (Foundational blockchain reference)
- TensorFlow Developers. (2024). TensorFlow: Large-Scale Machine Learning. tensorflow.org.
- FastAPI Documentation. (2024). FastAPI: Modern, fast web framework for building APIs with Python. fastapi.tiangolo.com.

# 18. Document Control & Sign-Off

| Field | Value |
|---|---|
| **Document Title** | TourSafe: AI-Driven Emergency Response & Blockchain Identity Infrastructure — Full Project Blueprint |
| **Document Version** | v1.0 |
| **Prepared By** | Vishal Lakshmikanthan, Sneha C & Madhu (TriArch Team) |
| **Institution** | Sri Sairam Engineering College, Chennai |
| **Department** | Computer Science & Engineering |
| **Date of Issue** | June 2026 |
| **Document Status** | Final — AI Agent Implementation Ready |
| **Intended Audience** | Development team, AI coding agents, academic supervisors, grant evaluators, hackathon judges |
| **Next Review Date** | Upon completion of Sprint 1 deliverables |

*End of TourSafe Project Blueprint Document*

TriArch Team | Sri Sairam Engineering College | 2026